

QUE1: Producer – Consumer

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 100

sem_t items;
pthread_mutex_t mutex;

int buffer[BUFFER_SIZE];
int fill_ptr = 0;
int use_ptr = 0;
int item_counter = 0;
int items_produced = 0;
int items_consumed = 0;

int produce_item() {
    return item_counter++;
}

void* producer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 100000; i++) {
        int item = produce_item();

        pthread_mutex_lock(&mutex);
        buffer[fill_ptr] = item;
        fill_ptr = (fill_ptr + 1) % BUFFER_SIZE;
        items_produced++;
        pthread_mutex_unlock(&mutex);
        printf("Producer %d produced: %d\n", id, item);
        sem_post(&items);
    }
    return NULL;
}

void* consumer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 100000; i++) {
        sem_wait(&items);
        pthread_mutex_lock(&mutex);
        int item = buffer[use_ptr];
        use_ptr = (use_ptr + 1) % BUFFER_SIZE;
        items_consumed++;
```

```

        pthread_mutex_unlock(&mutex);
        printf("Consumer %d consumed: %d\n", id, item);

    }
    return NULL;
}

int main() {
    pthread_t prod_threads[2], cons_threads[2];
    int prod_ids[2] = {1, 2};
    int cons_ids[2] = {1, 2};

    sem_init(&items, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    printf("Starting Producer-Consumer (BUGGY VERSION)\n");

    for(int i = 0; i < 2; i++) {
        pthread_create(&prod_threads[i], NULL, producer, &prod_ids[i]);
        pthread_create(&cons_threads[i], NULL, consumer, &cons_ids[i]);
    }

    for(int i = 0; i < 2; i++) {
        pthread_join(prod_threads[i], NULL);
        pthread_join(cons_threads[i], NULL);
    }

    sem_destroy(&items);
    pthread_mutex_destroy(&mutex);

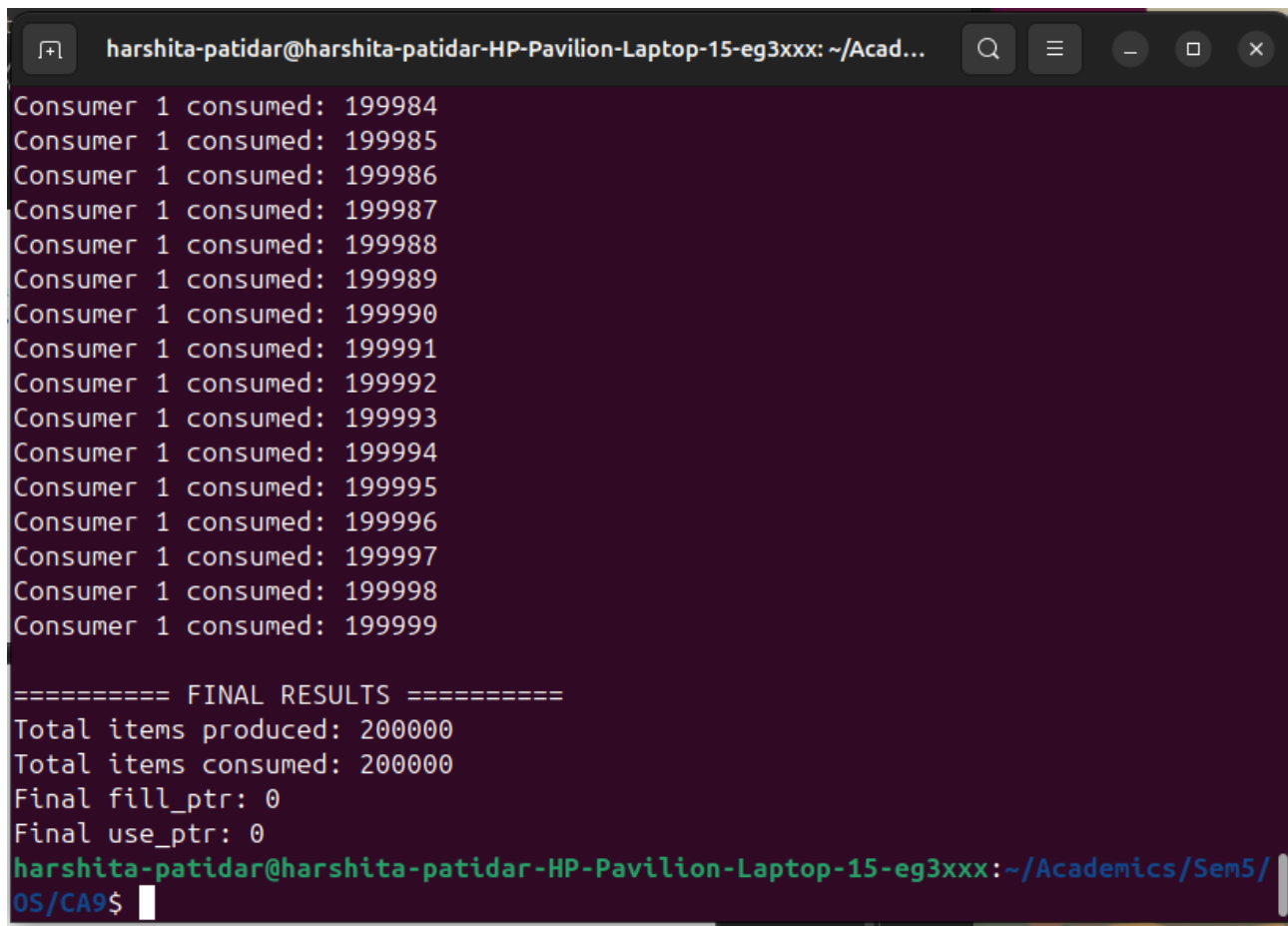
    printf("\n===== FINAL RESULTS =====\n");
    printf("Total items produced: %d\n", items_produced);
    printf("Total items consumed: %d\n", items_consumed);
    printf("Final fill_ptr: %d\n", fill_ptr);
    printf("Final use_ptr: %d\n", use_ptr);

    return 0;
}

```

Observation: Without a mutex, concurrent access to shared variables causes race conditions and data corruption; adding a mutex ensures mutual exclusion and consistent producer–consumer synchronization.

Screenshot:



```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Acad...
Consumer 1 consumed: 199984
Consumer 1 consumed: 199985
Consumer 1 consumed: 199986
Consumer 1 consumed: 199987
Consumer 1 consumed: 199988
Consumer 1 consumed: 199989
Consumer 1 consumed: 199990
Consumer 1 consumed: 199991
Consumer 1 consumed: 199992
Consumer 1 consumed: 199993
Consumer 1 consumed: 199994
Consumer 1 consumed: 199995
Consumer 1 consumed: 199996
Consumer 1 consumed: 199997
Consumer 1 consumed: 199998
Consumer 1 consumed: 199999

===== FINAL RESULTS =====
Total items produced: 200000
Total items consumed: 200000
Final fill_ptr: 0
Final use_ptr: 0
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/
OS/CA9$
```

QUE 2: Consumer – Producer (use two semaphore full and empty)

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 100

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

int buffer[BUFFER_SIZE];
int fill_ptr = 0;
int use_ptr = 0;
```

```

int item_counter = 0;
int items_produced = 0;
int items_consumed = 0;

int produce_item() {
    return item_counter++;
}

void* producer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 100000; i++) {
        int item = produce_item();

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[fill_ptr] = item;
        fill_ptr = (fill_ptr + 1) % BUFFER_SIZE;
        items_produced++;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        printf("Producer %d produced: %d\n", id, item);
    }
    return NULL;
}

void* consumer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 100000; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[use_ptr];
        use_ptr = (use_ptr + 1) % BUFFER_SIZE;
        items_consumed++;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        printf("Consumer %d consumed: %d\n", id, item);
    }
    return NULL;
}

int main() {
    pthread_t prod_threads[2], cons_threads[2];
    int prod_ids[2] = {1, 2};
    int cons_ids[2] = {1, 2};

```

```

sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full,0,0);
pthread_mutex_init(&mutex,NULL);

printf("Starting Producer-Consumer (BUGGY VERSION)\n");

for(int i = 0; i < 2; i++) {
    pthread_create(&prod_threads[i], NULL, producer, &prod_ids[i]);
    pthread_create(&cons_threads[i], NULL, consumer, &cons_ids[i]);
}

for(int i = 0; i < 2; i++) {
    pthread_join(prod_threads[i], NULL);
    pthread_join(cons_threads[i], NULL);
}

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

printf("\n===== FINAL RESULTS =====\n");
printf("Total items produced: %d\n", items_produced);
printf("Total items consumed: %d\n", items_consumed);
printf("Final fill_ptr: %d\n", fill_ptr);
printf("Final use_ptr: %d\n", use_ptr);

return 0;}

```

Screenshot:

```

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Acad...
Consumer 2 consumed: 199984
Consumer 2 consumed: 199985
Consumer 2 consumed: 199986
Consumer 2 consumed: 199987
Consumer 2 consumed: 199988
Consumer 2 consumed: 199989
Consumer 2 consumed: 199990
Consumer 2 consumed: 199991
Consumer 2 consumed: 199992
Consumer 2 consumed: 199993
Consumer 2 consumed: 199994
Consumer 2 consumed: 199995
Consumer 2 consumed: 199996
Consumer 2 consumed: 199997
Consumer 2 consumed: 199998
Consumer 2 consumed: 199999

===== FINAL RESULTS =====
Total items produced: 200000
Total items consumed: 200000
Final fill_ptr: 0
Final use_ptr: 0
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/
OS/CA9$

```

Observation:

By using two semaphores empty and full and a mutex, this version enforces buffer capacity limits, producers block when the buffer is full and consumers block when it's empty, achieving safe, bounded producer-consumer synchronization without data corruption.

Que 3:

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

sem_t empty;
sem_t full;
pthread_mutex_t mutex;
int buffer[BUFFER_SIZE];
int fill_ptr = 0;
int use_ptr = 0;
int item_counter = 0;

int produce_item() {
    return item_counter++;
}

void* producer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 10; i++) {
        int item = produce_item();

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[fill_ptr] = item;
        fill_ptr = (fill_ptr + 1) % BUFFER_SIZE;
        printf("Producer %d produced: %d\n", id, item);

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        usleep(100000);
    }
    return NULL;
}

void* consumer(void* arg) {
    int id = *(int*)arg;
```

```

for(int i = 0; i < 10; i++) {

    sem_wait(&full);
    pthread_mutex_lock(&mutex);

    int item = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % BUFFER_SIZE;
    printf("Consumer %d consumed: %d\n", id, item);

    pthread_mutex_unlock(&mutex);
    sem_post(&empty);
    usleep(150000);
}
return NULL;
}

int main() {
    pthread_t prod1, cons1;
    int prod_id = 1, cons_id = 1;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    printf("Starting code - WILL NOT DEADLOCK!\n");

    pthread_create(&prod1, NULL, producer, &prod_id);
    pthread_create(&cons1, NULL, consumer, &cons_id);

    pthread_join(prod1, NULL);
    pthread_join(cons1, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    printf("Program completed\n");

    return 0;
}

```

Observation:

Deadlock occurred because both threads locked the mutex before waiting on the semaphore; by acquiring semaphores before locking and using separate empty and full semaphores, mutual blocking is eliminated and correct synchronization is achieved.

Screenshot:

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA9
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA9$ gcc q3.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA9$ ./a.out

Starting code - WILL NOT DEADLOCK!
Producer 1 produced: 0
Consumer 1 consumed: 0
Producer 1 produced: 1
Consumer 1 consumed: 1
Producer 1 produced: 2
Consumer 1 consumed: 2
Producer 1 produced: 3
Producer 1 produced: 4
Consumer 1 consumed: 3
Producer 1 produced: 5
Consumer 1 consumed: 4
Producer 1 produced: 6
Producer 1 produced: 7
Consumer 1 consumed: 5
Producer 1 produced: 8
Consumer 1 consumed: 6
Producer 1 produced: 9
Consumer 1 consumed: 7
Consumer 1 consumed: 8
Consumer 1 consumed: 9
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA9$
```

Que 4: Reader – Writer bug

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t roomEmpty;
pthread_mutex_t mutex;

int readers = 0;
int shared_data = 0;
int read_count = 0;
int write_count = 0;

void* reader(void* arg) {
    int id = *(int*)arg;
    for (int i = 0; i < 50; i++) {

        pthread_mutex_lock(&mutex);
        readers++;
        if (readers == 1) {
            sem_wait(&roomEmpty);
        }
        pthread_mutex_unlock(&mutex);

        int value = shared_data;
```



```

    read_count++;
    printf("Reader %d reads: %d (active readers=%d)\n", id, value, readers);
    usleep(10000);

    pthread_mutex_lock(&mutex);
    readers--;
    if (readers == 0) {
        sem_post(&roomEmpty);
    }
    pthread_mutex_unlock(&mutex);

    usleep(10000);
}
return NULL;
}

void* writer(void* arg) {
    int id = *(int*)arg;
    for (int i = 0; i < 20; i++) {
        sem_wait(&roomEmpty);
        shared_data++;
        write_count++;
        printf("Writer %d writes: %d\n", id, shared_data);
        usleep(50000);
        sem_post(&roomEmpty);
        usleep(100000);
    }
    return NULL;
}

int main() {
    pthread_t reader_threads[4], writer_threads[2];
    int reader_ids[4] = {1, 2, 3, 4};
    int writer_ids[2] = {1, 2};

    sem_init(&roomEmpty, 0, 1);
    pthread_mutex_init(&mutex, NULL);

    printf("=== FIXED READER-WRITER VERSION ===\n");
    printf("Readers can read concurrently, writers have exclusive access.\n\n");

    for (int i = 0; i < 4; i++) {
        pthread_create(&reader_threads[i], NULL, reader, &reader_ids[i]);
    }
    for (int i = 0; i < 2; i++) {
        pthread_create(&writer_threads[i], NULL, writer, &writer_ids[i]);
    }

    for (int i = 0; i < 4; i++) {
        pthread_join(reader_threads[i], NULL);
    }
    for (int i = 0; i < 2; i++) {

```

```

    pthread_join(writer_threads[i], NULL);
}

sem_destroy(&roomEmpty);
pthread_mutex_destroy(&mutex);

printf("\n===== RESULTS =====\n");
printf("Final readers counter: %d (should be 0)\n", readers);
printf("Total reads: %d\n", read_count);
printf("Total writes: %d (expected: 40)\n", write_count);
printf("Final shared_data: %d (expected: 40)\n", shared_data);

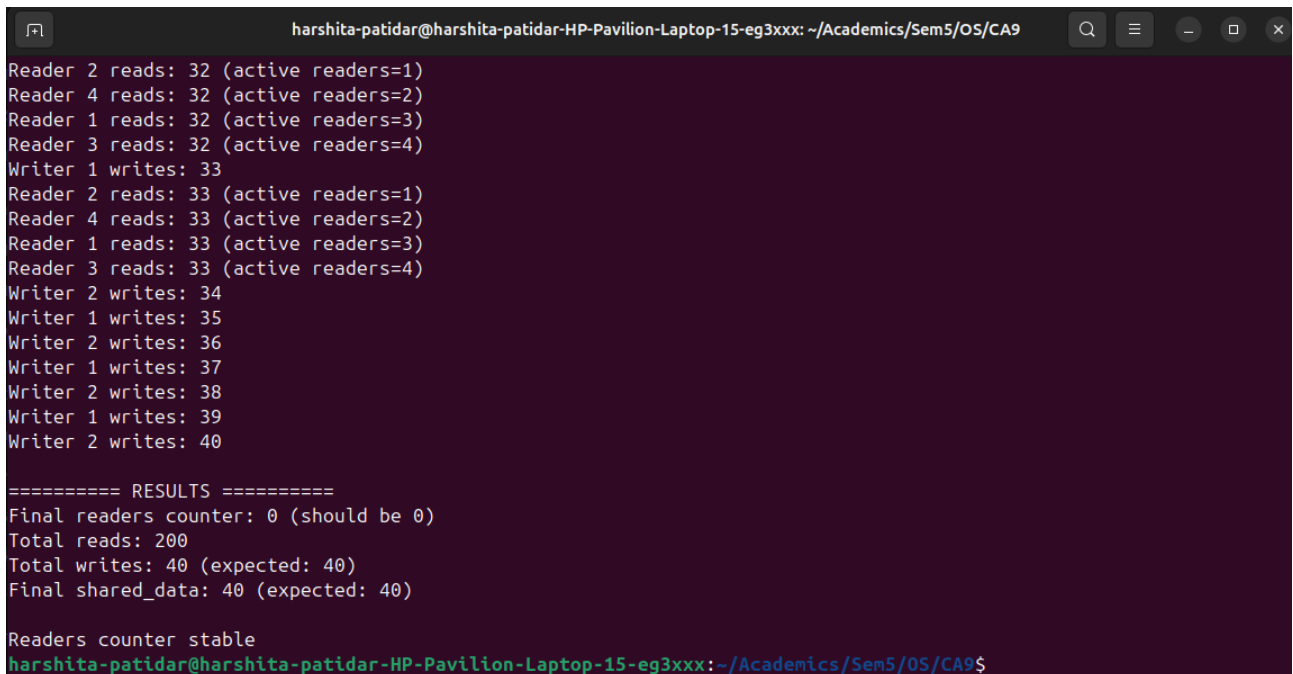
if (readers != 0) {
    printf("\nBUG DETECTED: readers counter is corrupted!\n");
} else {
    printf("\nReaders counter stable\n");
}
return 0;
}

```

Observation:

bug occurred because multiple readers modified the shared readers counter concurrently, causing incorrect synchronization with writers; adding a mutex around readers updates ensures atomicity and prevents data corruption.

Screenshot:



```

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA9
Reader 2 reads: 32 (active readers=1)
Reader 4 reads: 32 (active readers=2)
Reader 1 reads: 32 (active readers=3)
Reader 3 reads: 32 (active readers=4)
Writer 1 writes: 33
Reader 2 reads: 33 (active readers=1)
Reader 4 reads: 33 (active readers=2)
Reader 1 reads: 33 (active readers=3)
Reader 3 reads: 33 (active readers=4)
Writer 2 writes: 34
Writer 1 writes: 35
Writer 2 writes: 36
Writer 1 writes: 37
Writer 2 writes: 38
Writer 1 writes: 39
Writer 2 writes: 40

===== RESULTS =====
Final readers counter: 0 (should be 0)
Total reads: 200
Total writes: 40 (expected: 40)
Final shared_data: 40 (expected: 40)

Readers counter stable
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA9$

```

Que 5

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

typedef struct {
    int counter;
    sem_t mutex;
} Lightswitch;

Lightswitch readSwitch;
sem_t roomEmpty;
int shared_data = 0;

void lightswitch_init(Lightswitch *ls) {
    ls->counter = 0;
    sem_init(&ls->mutex, 0, 1);
}

void lightswitch_lock(Lightswitch *ls, sem_t *semaphore) {
    sem_wait(&ls->mutex);
    ls->counter++;
    if (ls->counter == 1) {
        sem_wait(semaphore);
    }
    sem_post(&ls->mutex);
}

void lightswitch_unlock(Lightswitch *ls, sem_t *semaphore) {
    sem_wait(&ls->mutex);
    ls->counter--;
    if (ls->counter == 0) {
        sem_post(semaphore);
    }
    sem_post(&ls->mutex);
}

void *reader(void *arg) {
    int id = *(int *)arg;

    lightswitch_lock(&readSwitch, &roomEmpty);

    printf("Reader %d is reading...\n", id);
    sleep(1);
    printf("Reader %d finished reading.\n", id);
}
```

```

    lightswitch_unlock(&readSwitch, &roomEmpty);

    return NULL;
}

void *writer(void *arg) {
    int id = *(int *)arg;

    sem_wait(&roomEmpty);

    printf("Writer %d is writing...\n", id);
    shared_data++;
    sleep(2);
    printf("Writer %d finished writing.\n", id);

    sem_post(&roomEmpty);

    return NULL;
}

int main() {
    pthread_t r1, r2, w1, w2;
    int rID1 = 1, rID2 = 2;
    int wID1 = 1, wID2 = 2;

    lightswitch_init(&readSwitch);
    sem_init(&roomEmpty, 0, 1);

    pthread_create(&r1, NULL, reader, &rID1);
    pthread_create(&r2, NULL, reader, &rID2);
    pthread_create(&w1, NULL, writer, &wID1);
    pthread_create(&w2, NULL, writer, &wID2);

    pthread_join(r1, NULL);
    pthread_join(r2, NULL);
    pthread_join(w1, NULL);
    pthread_join(w2, NULL);

    sem_destroy(&roomEmpty);
    sem_destroy(&readSwitch.mutex);

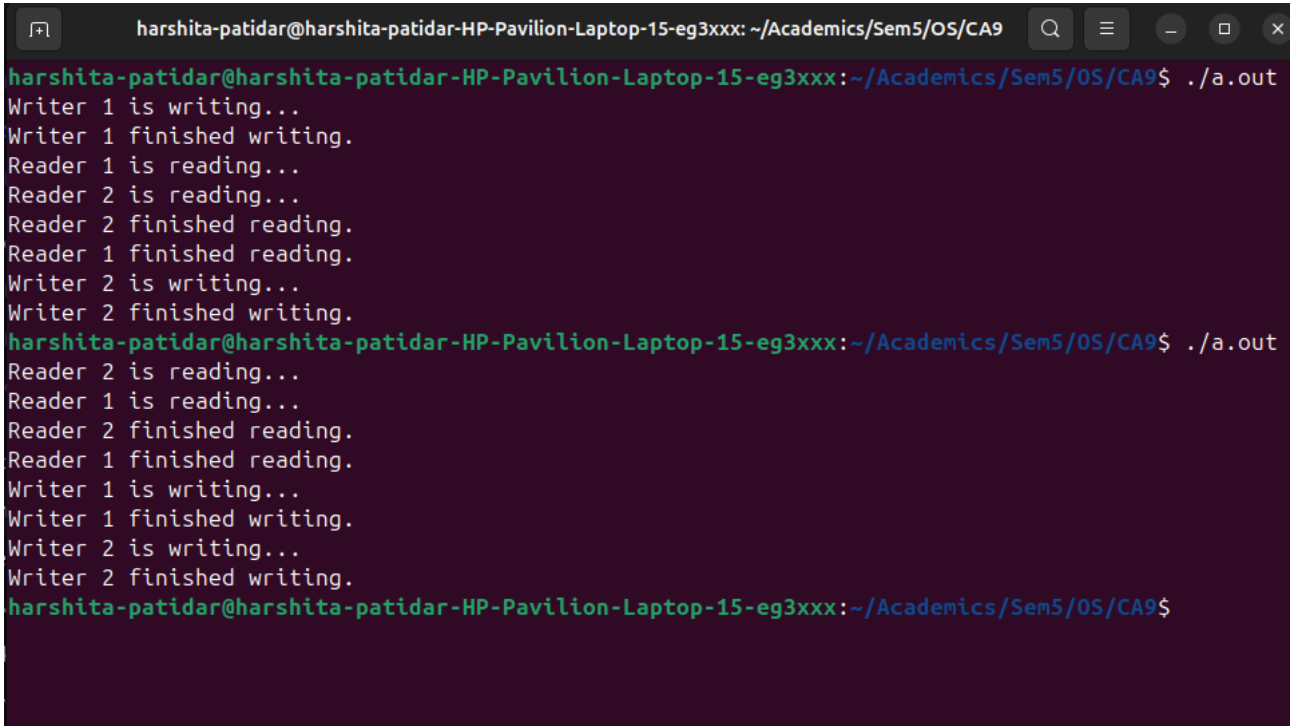
    return 0;
}

```

Observation:

This implementation ensures that multiple readers read concurrently while writers still get exclusive access, preventing data corruption and starvation.

Screenshot:



```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA9$ ./a.out
Writer 1 is writing...
Writer 1 finished writing.
Reader 1 is reading...
Reader 2 is reading...
Reader 2 finished reading.
Reader 1 finished reading.
Writer 2 is writing...
Writer 2 finished writing.
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA9$ ./a.out
Reader 2 is reading...
Reader 1 is reading...
Reader 2 finished reading.
Reader 1 finished reading.
Writer 1 is writing...
Writer 1 finished writing.
Writer 2 is writing...
Writer 2 finished writing.
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA9$
```

Que 6:

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t forks[N];
int eat_count[N] = {0};

void* philosopher(void* arg) {
    int id = *(int*)arg;
    int left_fork = id;
    int right_fork = (id + 1) % N;

    for(int i = 0; i < 3; i++) {
        printf("Philosopher %d is thinking...\n", id);
        usleep(100000);

        printf("Philosopher %d is hungry, reaching for forks %d and %d\n", id, left_fork, right_fork);
```

```

int first = left_fork < right_fork ? left_fork : right_fork;
int second = left_fork < right_fork ? right_fork : left_fork;

// Pick forks in order of their IDs (lower first)
sem_wait(&forks[first]);
printf(" Philosopher %d picked up fork %d\n", id, first);
usleep(50000);

sem_wait(&forks[second]);
printf(" Philosopher %d picked up fork %d\n", id, second);

printf("Philosopher %d is EATING (meal #%d)\n", id, eat_count[id] + 1);
eat_count[id]++;
usleep(200000);

sem_post(&forks[first]);
sem_post(&forks[second]);
printf(" Philosopher %d put down both forks\n\n", id);
}

return NULL;
}

int main() {
    pthread_t philosophers[N];
    int ids[N];

    for(int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1);
        ids[i] = i;
    }

    printf("=== DINING PHILOSOPHERS - FIXED (LOWER ID FIRST) ===\n\n");

    for(int i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for(int i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }

    for(int i = 0; i < N; i++) {
        sem_destroy(&forks[i]);
    }

    printf("\n===== MEAL COUNT =====\n");
    for(int i = 0; i < N; i++) {
        printf("Philosopher %d ate %d times (expected: 3)\n", i, eat_count[i]);
    }

    return 0;
}

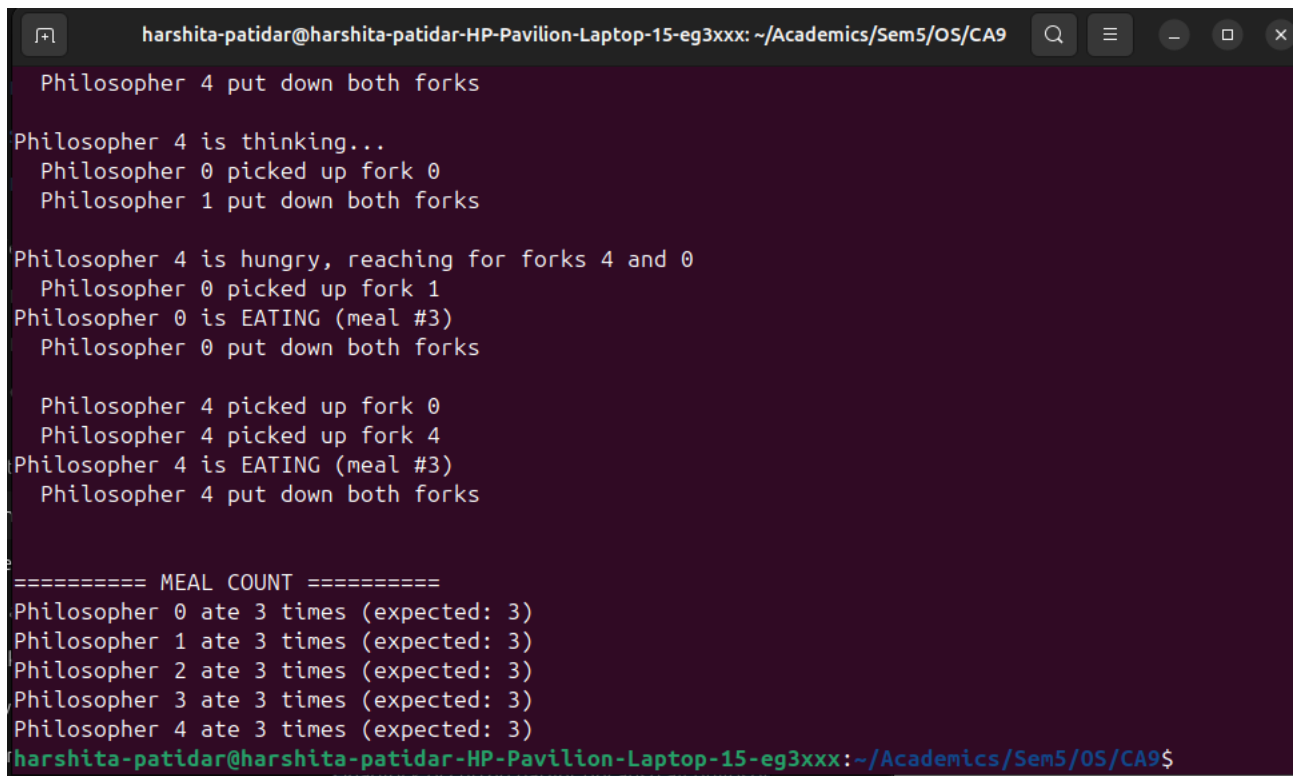
```

}

Observation:

Deadlock occurred earlier because all philosophers waited for each other's right fork (circular wait). By picking up the lower-numbered fork first, we impose a strict order, removing circular dependency and eliminating deadlock.

Screenshot:



```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA9
Philosopher 4 put down both forks
Philosopher 4 is thinking...
Philosopher 0 picked up fork 0
Philosopher 1 put down both forks
Philosopher 4 is hungry, reaching for forks 4 and 0
Philosopher 0 picked up fork 1
Philosopher 0 is EATING (meal #3)
Philosopher 0 put down both forks
Philosopher 4 picked up fork 0
Philosopher 4 picked up fork 4
Philosopher 4 is EATING (meal #3)
Philosopher 4 put down both forks

===== MEAL COUNT =====
Philosopher 0 ate 3 times (expected: 3)
Philosopher 1 ate 3 times (expected: 3)
Philosopher 2 ate 3 times (expected: 3)
Philosopher 3 ate 3 times (expected: 3)
Philosopher 4 ate 3 times (expected: 3)
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA9$
```